



Documentação Técnica do Front-End: Conecta à Vida

Painel de Controle Administrativo em React, TypeScript e Vite



1. Tema do Aplicativo

O front-end do **Conecta à Vida** é uma aplicação web do tipo **SPA (Single Page Application)** focada na consolidação visual, gerenciamento de dados de saúde pública e transmissão instantânea de comunicados governamentais. A interface funciona como a cabine de comando do administrador do sistema, permitindo monitorar crises e gerir a infraestrutura médica local.



2. Detalhamento do Tema

Construída com **React** e **TypeScript**, a interface foi projetada sob o conceito de **Design Semântico**, onde cores e formas indicam o nível de gravidade das informações (ex: vermelho para emergências, verde para ações preventivas).

O ecossistema consome de forma totalmente assíncrona uma API REST em Spring Boot. Utilizando a biblioteca de estilização **Tailwind CSS** e componentes de alta fidelidade visual (inspirados no **Shadcn UI**), o painel garante uma experiência de uso fluida, responsiva e performática, minimizando o tempo de resposta do gestor em situações de risco à saúde coletiva.



3. Funcionalidades e Casos de Uso

A interface do usuário está estruturada em módulos operacionais claros:

A. Autenticação e Barreiras de Proteção

- **Caso de Uso (Login):** O usuário digita suas credenciais corporativas. O React submete uma requisição POST assíncrona. Em caso de sucesso, o objeto do usuário é salvo no localStorage do navegador sob a chave @conecta:admin e a tela é redirecionada.
- **Caso de Uso (Bloqueio Malicioso):** Se um usuário tentar digitar a URL de uma página interna diretamente na barra de endereços sem estar logado, o componente RotaProtegida intercepta a navegação, limpa o histórico e o redireciona de volta para a tela de login.

B. Painel de Indicadores (Dashboard)

- **Caso de Uso (Monitoramento Paralelo):** Ao carregar a página inicial, o React dispara requisições assíncronas paralelas via Promise.all para coletar dados volumetricos, coordenadas do gráfico e a trilha de auditoria.
- **Caso de Uso (Leitura de Dados):** O administrador analisa os 4 cards superiores de métricas, avalia a curva de crescimento de novos usuários através do gráfico de barras interativo e monitora as últimas 5 ações técnicas através da timeline de auditoria em tempo real.

C. Módulo de Cidadãos (Usuários)

- **Caso de Uso (Filtragem Dinâmica):** O gestor digita um termo no campo de busca e o React, através do Hook `useMemo`, filtra a tabela instantaneamente na memória, sem sobrecarregar o banco de dados.
- **Caso de Uso (Carga em Massa):** O administrador anexa uma planilha `.csv` no input. A interface empacota o arquivo físico usando um objeto `FormData` e faz o upload via `multipart`. O sistema exibe um alerta flutuante (*Toast*) confirmando o número de registros importados com sucesso.
- **Caso de Uso (Extração Documental):** O usuário clica em "Exportar PDF" ou "Exportar CSV", abrindo uma nova aba que consome os streams de arquivos diretamente da API.

D. Unidades de Saúde e Postos de Atendimento

- **Caso de Uso (Manutenção da Infraestrutura):** Permite listar, cadastrar e atualizar hospitais, UPAs e UBSs locais através de formulários inteligentes embarcados em modais dinâmicos (*Dialog*), incluindo a definição dos cronogramas de atendimento para dias úteis e finais de semana.

E. Emissão de Alertas Emergenciais e Campanhas

- **Caso de Uso (Disparo de Alarme):** O administrador preenche o formulário de risco (definindo título, descrição, gravidade e bairro afetado). O React envia o JSON para o banco, salvando o registro com a flag `lido = false`, o que ativa imediatamente o alerta no aplicativo dos cidadãos.
- **Caso de Uso (Controle de Mutirões):** Criação e gerenciamento de campanhas de vacinação ou doação de sangue com controle visual de status e datas cronológicas de vigência.

4. Estrutura Visual, Organização da Interface e Fluxo de Navegação

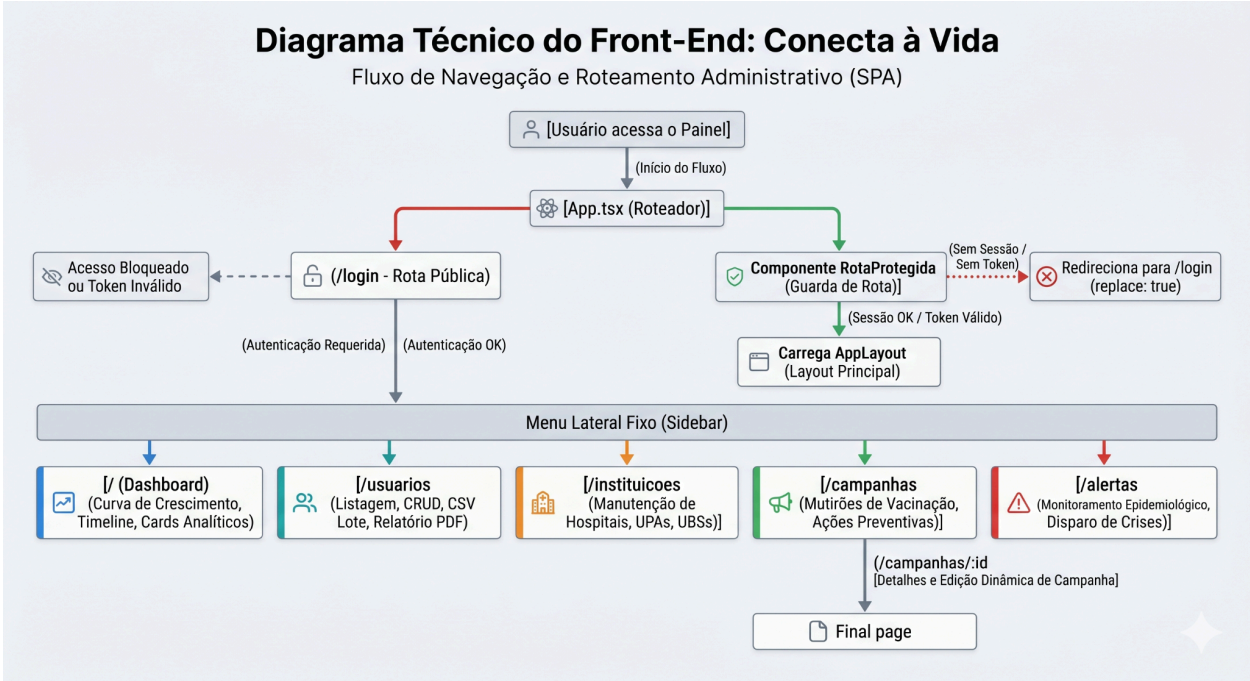
Organização do Projeto (src/)

A arquitetura do código adota o padrão de separação por responsabilidades:

- **src/services/api.ts:** Camada isolada de rede. Contém os contratos (*Interfaces*) do TypeScript e as funções que realizam as chamadas HTTP para o backend.
- **src/components/AppLayout.tsx:** Componente estrutural fixo da aplicação que renderiza a barra lateral de navegação (*Sidebar*) e o cabeçalho de status administrativo.
- **src/pages/:** Contém as telas completas e isoladas da aplicação (*Login*, *Index*, *Usuarios*, *Instituicoes*, *Campanhas*, *CampanhaDetalhes*, *Alertas*).
- **src/App.tsx:** Roteador central da aplicação. Configura o comportamento de **Lazy Loading** (as páginas só são baixadas pelo navegador quando o usuário clica nelas) e aplica a barreira de segurança de rotas privadas.

 Fluxo de Navegação

O mapa abaixo ilustra o fluxo de trânsito do usuário pelas rotas da aplicação:



 5. Estrutura de Banco de Dados (Mapeamento no Front-End)

No front-end, o banco de dados do Supabase é representado através de **Interfaces do TypeScript**. Elas funcionam como contratos estritos que garantem que o dado manipulado nas telas possua exatamente o mesmo nome e tipo de coluna que existe na base de dados PostgreSQL.

Tabela de Interfaces e Mapeamento de Tipos

Interface TypeScript	Atributos Mapeados	Tipo de Dado	Correspondência no Banco
Usuario	id	number (opcional)	Chave Primária
	nome	string	Nome do cidadão
	email	string	E-mail (Unique)
	senha	string (opcional)	Senha

	idade	number (opcional)	Idade
	sexo	string (opcional)	Gênero
	localizacao	string (opcional)	Cargo (ex: 'Administrador')
InstituicaoSaude	id	number (opcional)	Chave Primária
	tipoInstituicao	string	Tipo ('HOSPITAL', 'UPA', 'UNIDADE')
	nome	string	Nome Fantasia
	email	string	E-mail
	telefone	string	Telefone
	endereco	string	Endereço físico
	horarioSegSex	string	Horário útil
	horarioSabado	string	Horário sábado
	horarioDomingo	string	Horário domingo
Alerta	id	number (opcional)	Chave Primária
	tipo	literal 'ALERTA'	Tipo de comunicação fixa
	categoria	string	Gravidade
	titulo	string	('urgente', 'aviso')
	descricao	string	Título do risco
	localizacao	string	Instruções de proteção
	lido	boolean	Bairro afetado
	dataPostada	string	

			Flag de controle ativo
			Data de envio
Campanha	id	number (opcional)	Chave Primária
	titulo	string	Título da ação
	descricao	string	Detalhes do mutirão
	categoria	string	Tipo ('Vacinação', etc)
	publicoAlvo	string	Grupo Alvo ('Idosos')
	status	string	Estado ('Ativa', 'Encerrada')
	dataInicio	string	Data abertura
	dataFim	string	Data encerramento
	linkimagem	string	URL da imagem em nuvem
LogAtividade	id	number	Chave Primária
	usuario	Usuario	Objeto completo do autor
	acao	string	Descrição da tarefa
	dataHora	string	Data e hora do log

🔗 **Nota técnica para a equipe:** O uso do caractere ? ao lado de atributos como id?: number nas interfaces do TypeScript é fundamental. Ele indica que o campo é opcional na memória do React. Isso permite que usemos a mesma interface para **criar** um registro (onde o ID ainda não existe e será gerado pelo auto-incremento do Supabase) e para **exibir/atualizar** um registro (onde o ID já vem preenchido do banco).